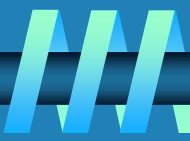
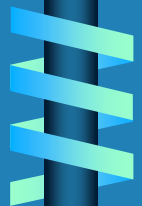
Tackling Difficult Combinatorial Problems



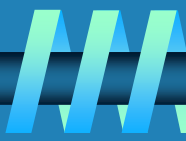
There are two principal approaches to tackling difficult combinatorial problems (NP-hard problems):

- Use a strategy that guarantees solving the problem exactly but doesn't guarantee to find a solution in polynomial time
- Use an approximation algorithm that can find an approximate (sub-optimal) solution in polynomial time



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Exact Solution Strategies

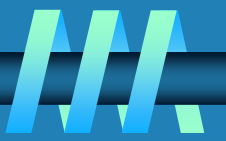


- *exhaustive search* (brute force)
  - useful only for small instances
- *dynamic programming*
  - applicable to some problems (e.g., the knapsack problem)
- *backtracking*
  - eliminates some unnecessary cases from consideration
  - yields solutions in reasonable time for many instances but worst case is still exponential
- *branch-and-bound*
  - further refines the backtracking idea for optimization problems



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

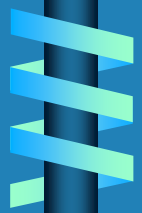
Backtracking



- Construct the *state-space tree*
  - nodes: partial solutions
  - edges: choices in extending partial solutions

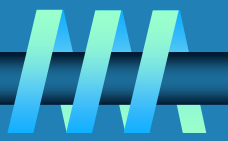
- Explore the state space tree using depth-first search

- "Prune" *nonpromising nodes*
  - stop exploring subtrees rooted at nodes that cannot lead to a solution and backtracks to such a node's parent to continue the search

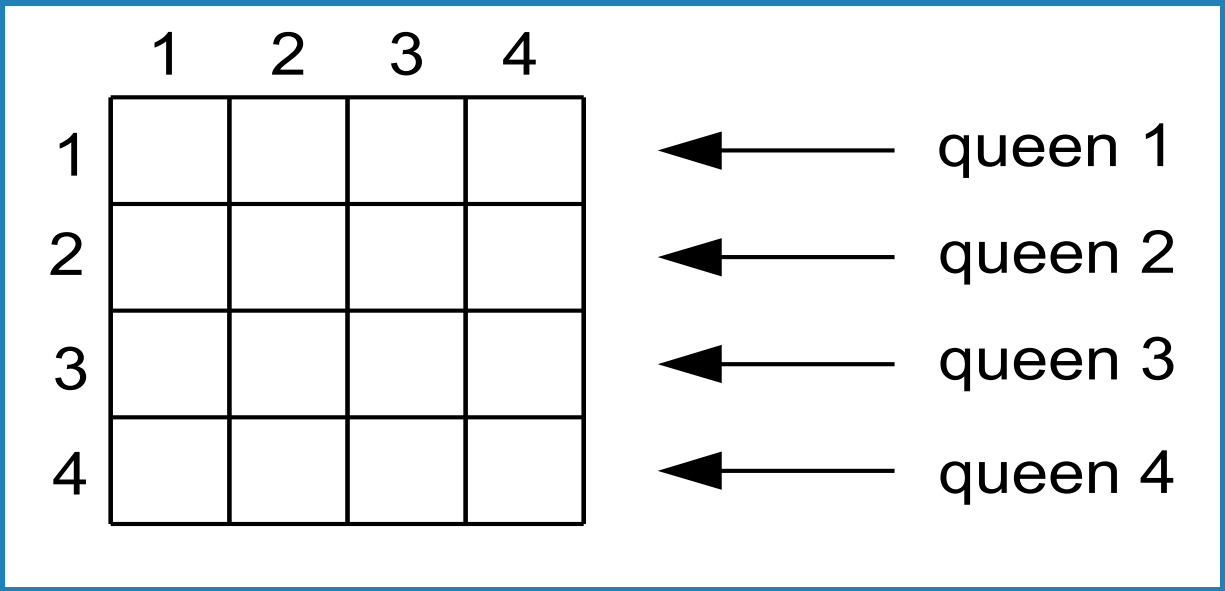


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: *n*-Queens Problem



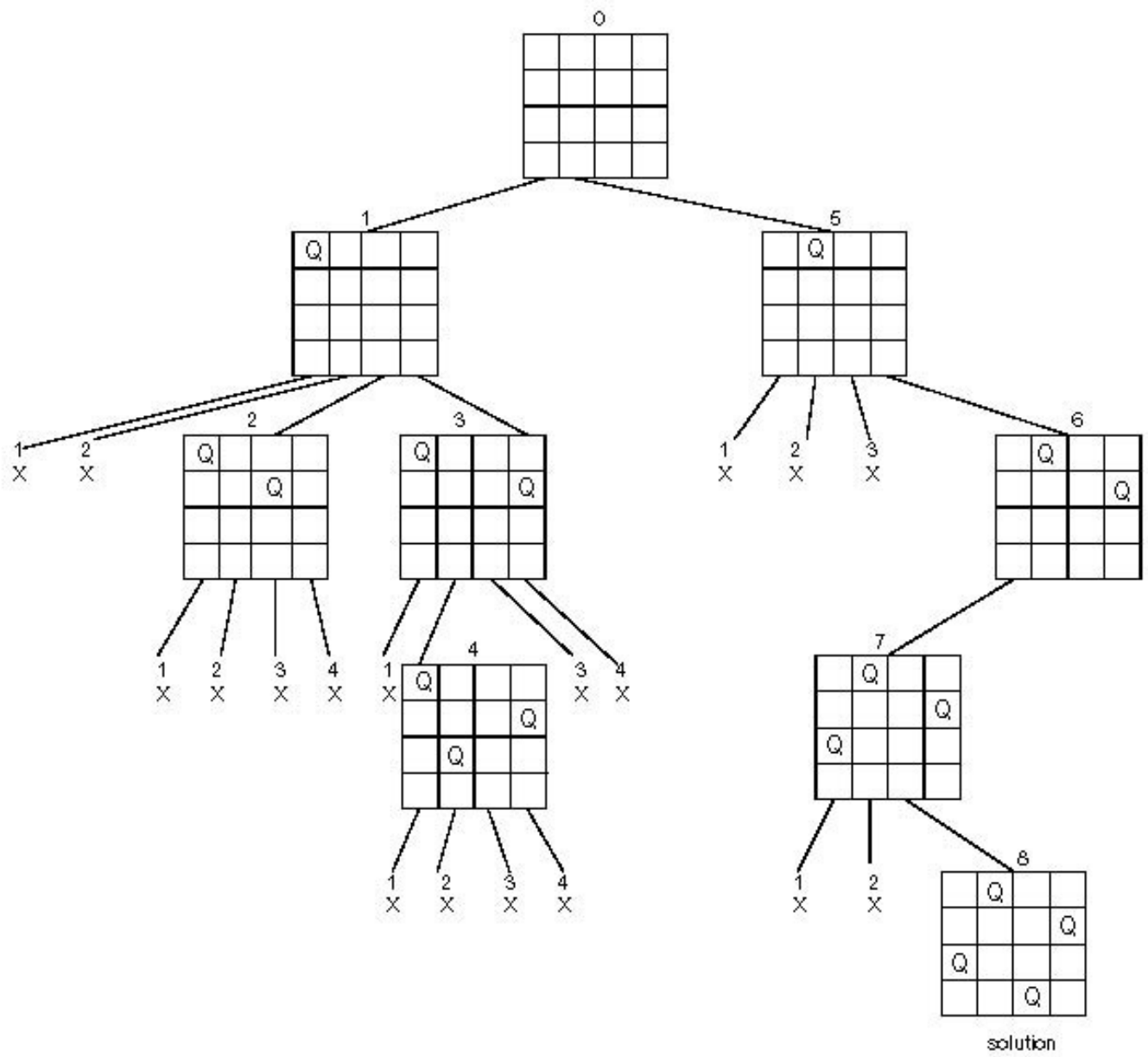
Place *n* queens on an *n*-by-*n* chess board so that no two of them are in the same row, column, or diagonal



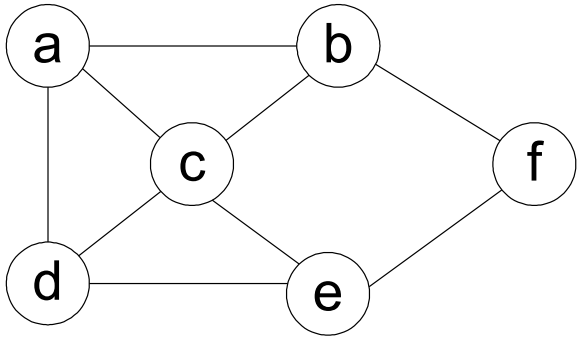


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

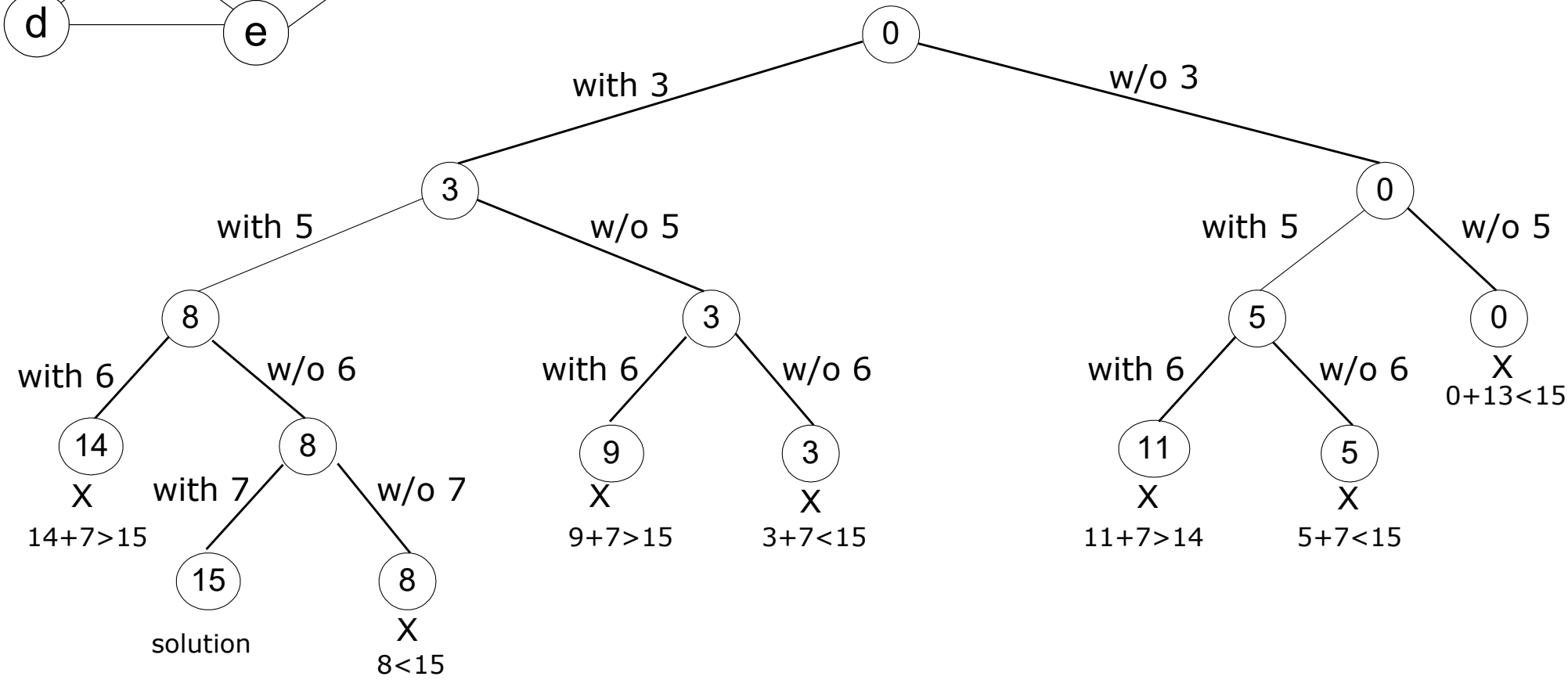
State-Space Tree of the 4-Queens Problem



Example: Hamiltonian Circuit Problem

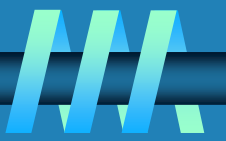


Subset-Sum $A = \{3, 5, 6, 7\}$ $Sum = 15$

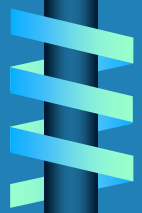


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Branch-and-Bound

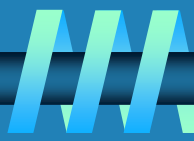


- An enhancement of backtracking
- Applicable to optimization problems
- For each node (partial solution) of a state-space tree, computes a bound on the value of the objective function for all descendants of the node (extensions of the partial solution)
- Uses the bound for:
  - ruling out certain nodes as "nonpromising" to prune the tree – if a node's bound is not better than the best solution seen so far
  - guiding the search through state-space



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: Assignment Problem



Select one element in each row of the cost matrix *C* so that:

- **no two selected elements are in the same column**
- **the sum is minimized**

Example

	Job 1	Job 2	Job 3	Job 4
Person *a*	9	2	7	8
Person *b*	6	4	3	7
Person *c*	5	8	1	8
Person *d*	7	6	9	4

Lower bound: Any solution to this problem will have total cost at least: 2 + 3 + 1 + 4 (or 5 + 2 + 1 + 4)



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: First two levels of the state-space tree

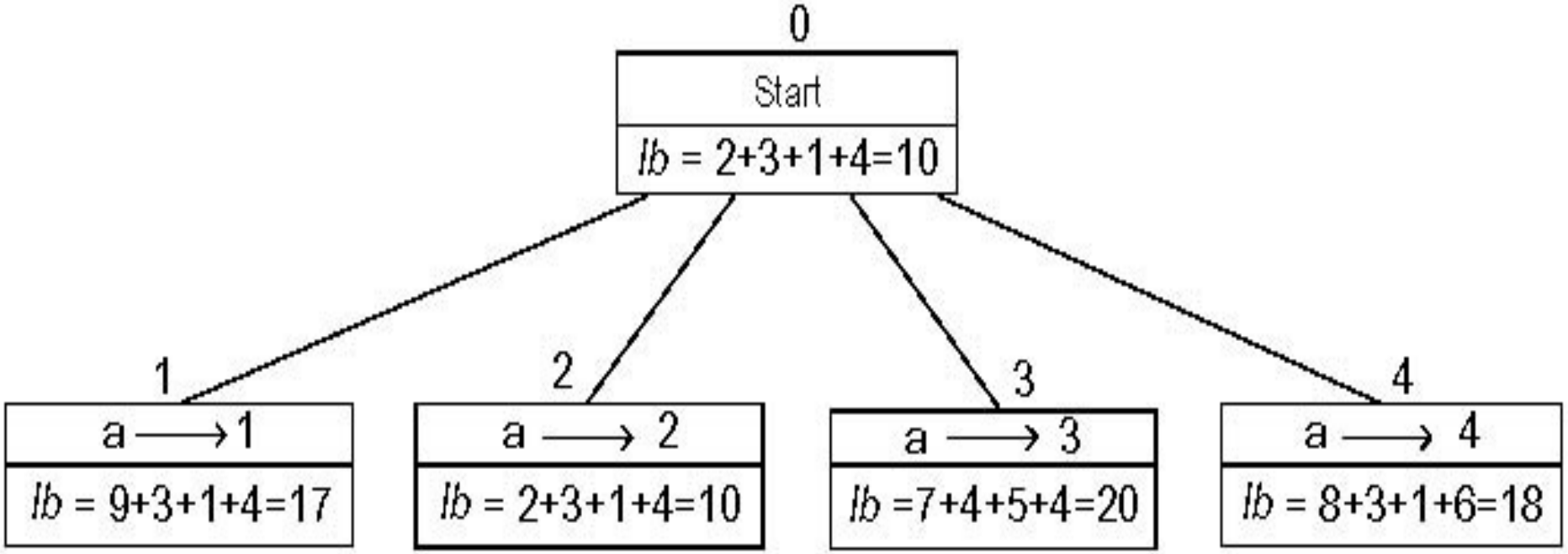


Figure Levels 0 and 1 of the state-space tree for the instance of the assignment problem being solved with the best-first branch-and-bound algorithm. The number above a node shows the order in which the node was generated. A node's fields indicate the job number assigned to person a and the lower bound value, lb, for this node.

Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example (cont.)

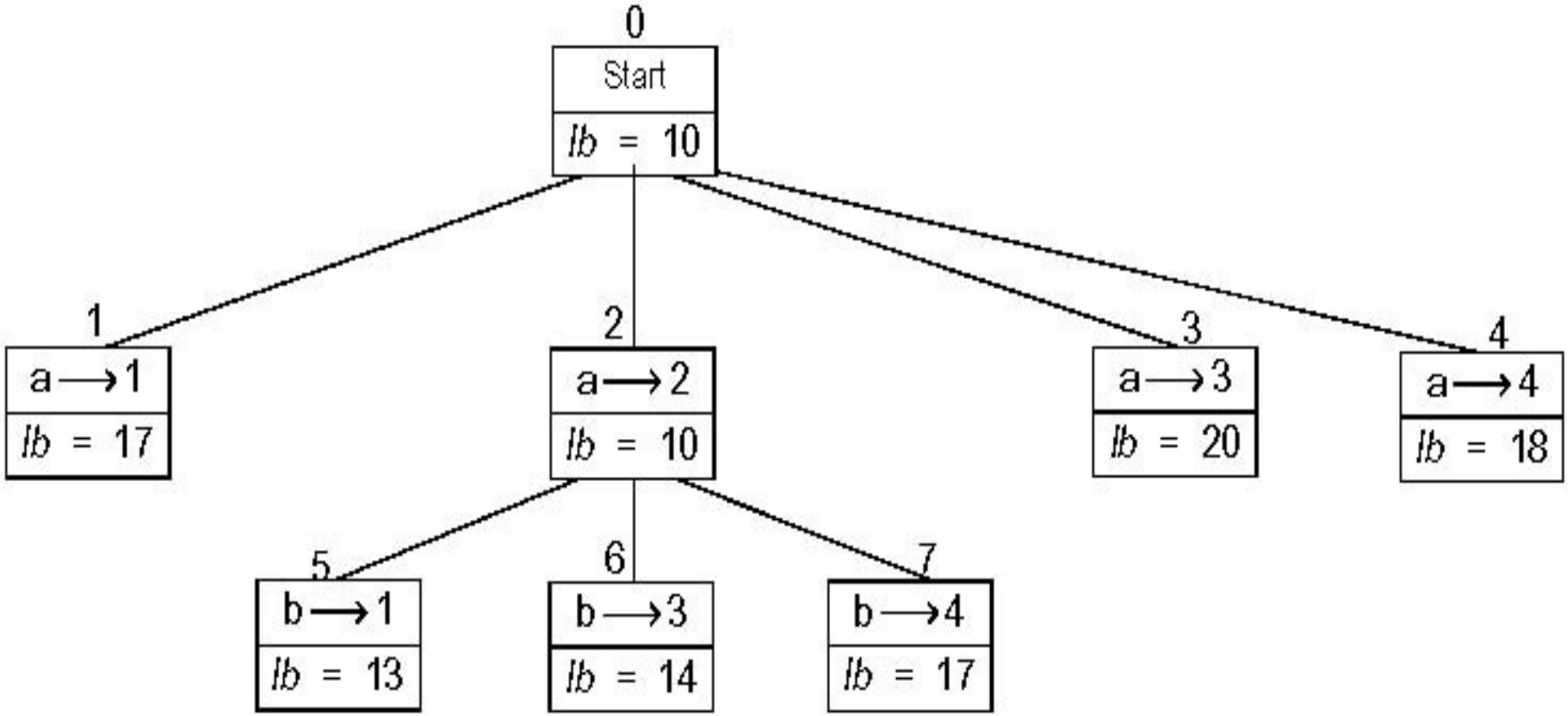


Figure Levels 0, 1, and 2 of the state-space tree for the instance of the assignment problem being solved with the best-first branch-and-bound algorithm

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: Complete state-space tree

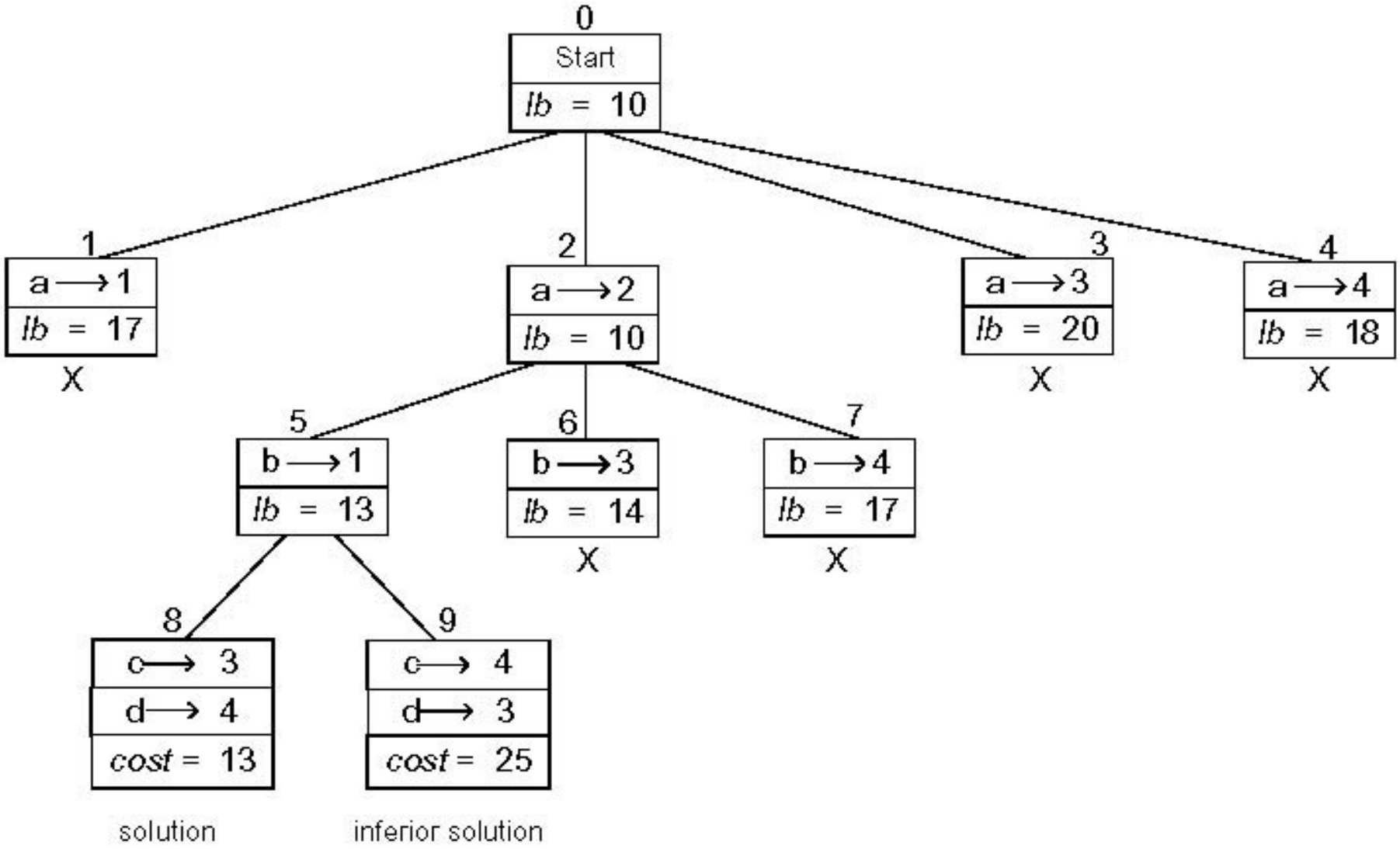
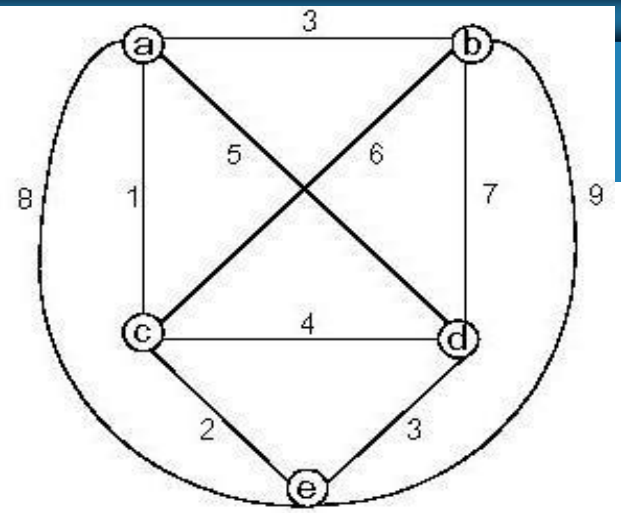


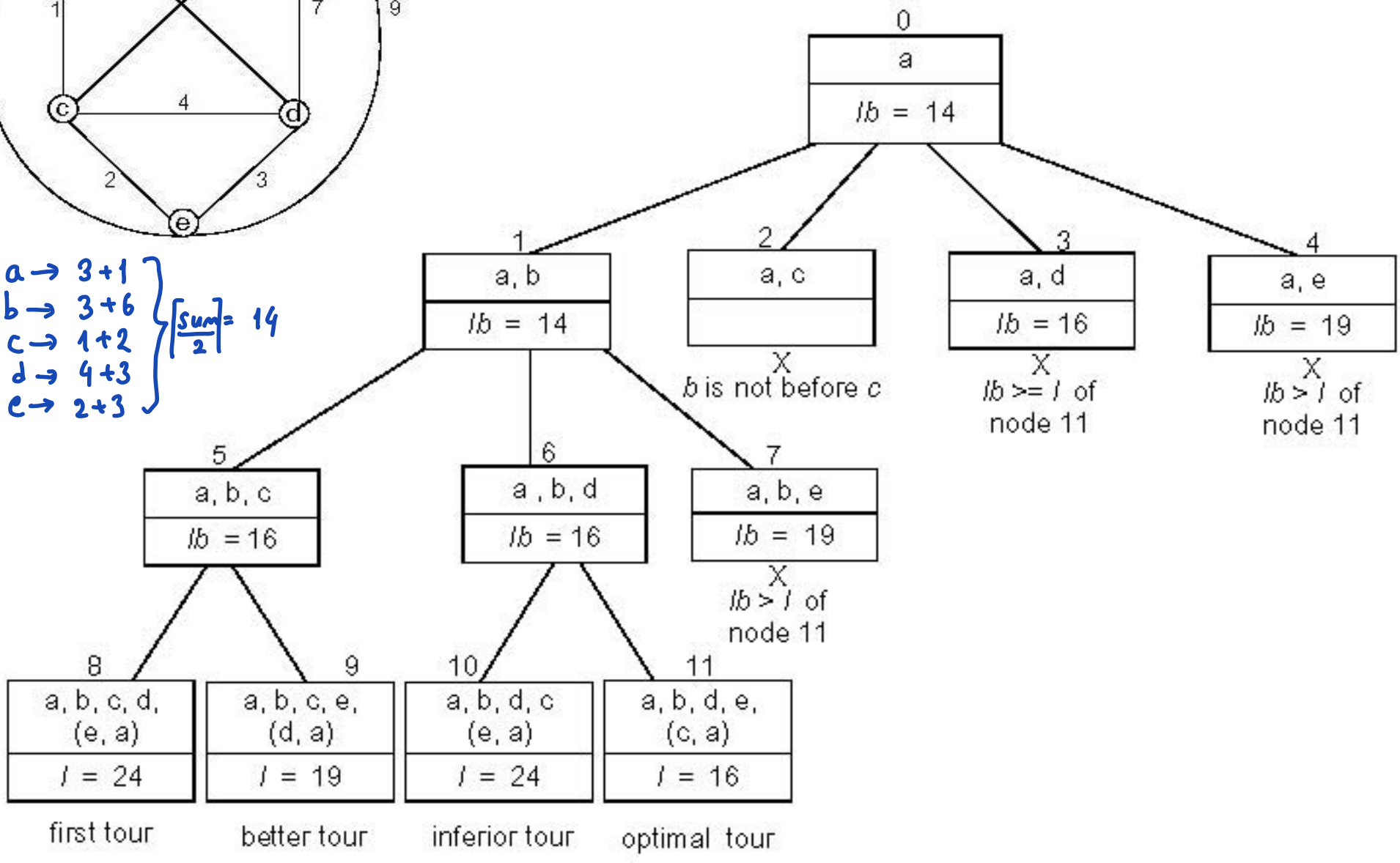
Figure Complete state-space tree for the instance of the assignment problem solved with the best-first branch-and-bound algorithm

Example: Traveling Salesman Problem

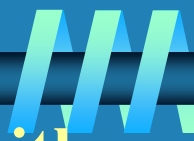


a → 3+1
b → 3+6
c → 1+2
d → 4+3
e → 2+3

$\left\lceil \frac{\text{sum}}{2} \right\rceil = 14$



Approximation Approach



Apply a fast (i.e., a polynomial-time) approximation algorithm to get a solution that is not necessarily optimal but hopefully close to it

Accuracy measures:

accuracy ratio **of an approximate solution s_a**

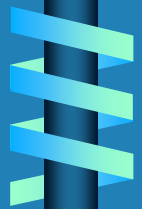
$r(s_a) = f(s_a) / f(s^*)$ **for minimization problems**

$r(s_a) = f(s^*) / f(s_a)$ **for maximization problems**

where $f(s_a)$ and $f(s^*)$ are values of the objective function f for the approximate solution s_a and actual optimal solution s^*

performance ratio **of the algorithm A**

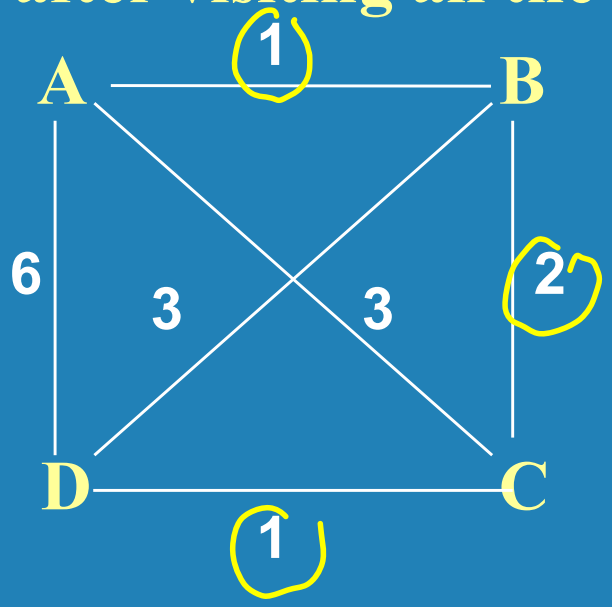
the lowest upper bound of $r(s_a)$ on all instances



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Nearest-Neighbor Algorithm for TSP

Starting at some city, always go to the nearest unvisited city, and, after visiting all the cities, return to the starting one



s_a : A – B – C – D – A of length 10

s^* : A – B – D – C – A of length 8

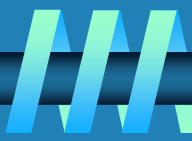
accuracy ratio = $\frac{10}{8} = 1.25$

Note: Nearest-neighbor tour may depend on the starting city

Accuracy: $R_A = \infty$ (unbounded above) – make the length of AD arbitrarily large in the above example

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

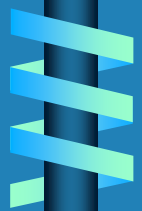
Multifragment-Heuristic Algorithm



Stage 1: Sort the edges in nondecreasing order of weights. Initialize the set of tour edges to be constructed to empty set

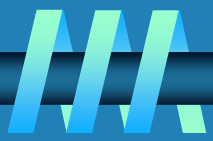
Stage 2: Add next edge on the sorted list to the tour, skipping those whose addition would've created a vertex of degree 3 or a cycle of length less than n. Repeat this step until a tour of length n is obtained

Note: $R_A = \infty$, but this algorithm tends to produce better tours than the nearest-neighbor algorithm



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Twice-Around-the-Tree Algorithm



Stage 1: Construct a minimum spanning tree of the graph (e.g., by Prim's or Kruskal's algorithm)

Stage 2: Starting at an arbitrary vertex, create a path that goes twice around the tree and returns to the same vertex

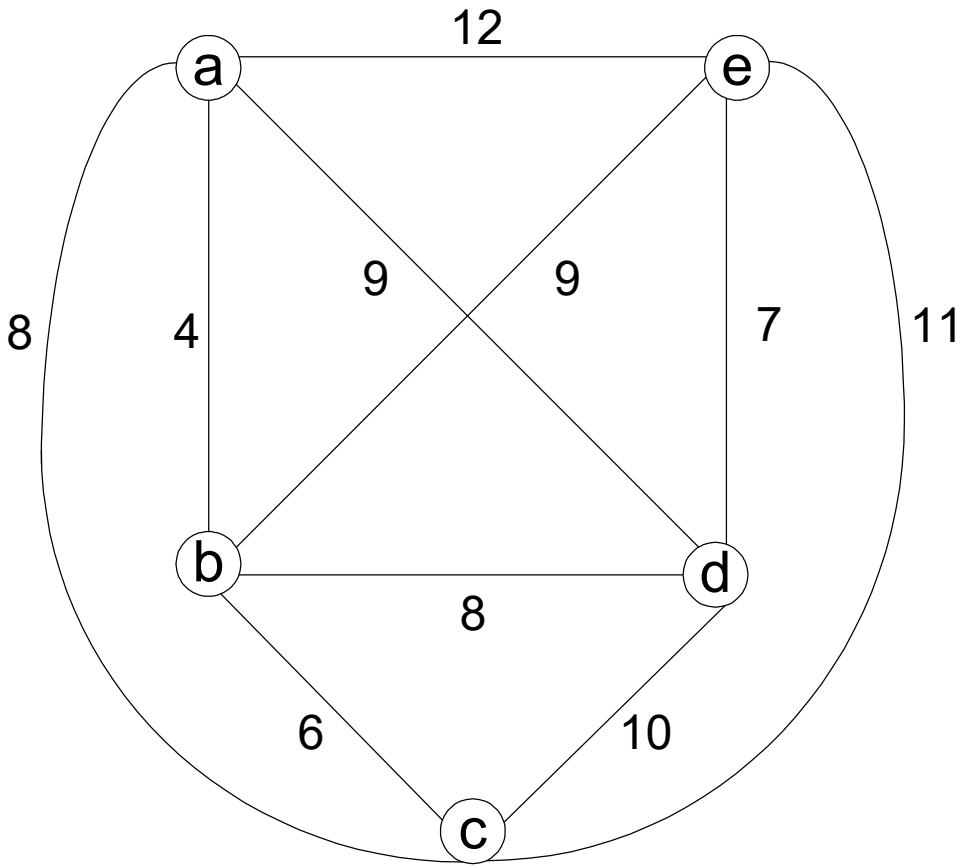
Stage 3: Create a tour from the circuit constructed in Stage 2 by making shortcuts to avoid visiting intermediate vertices more than once

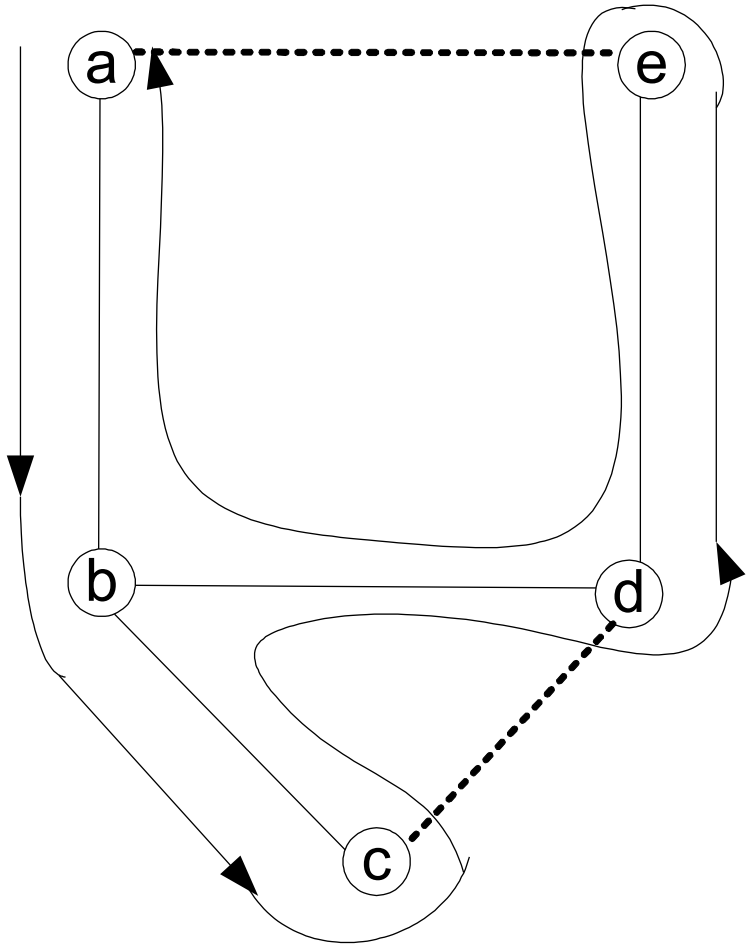
Note: $R_A = \infty$ for general instances, but this algorithm tends to produce better tours than the nearest-neighbor algorithm



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example



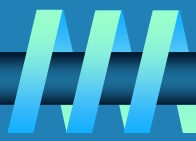


Walk: a – b – c – b – d – e – d – b – a

Tour: a – b – c – d – e – a

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Christofides Algorithm



Stage 1: Construct a minimum spanning tree of the graph

Stage 2: Add edges of a minimum-weight matching of all the odd vertices in the minimum spanning tree

Stage 3: Find an Eulerian circuit of the multigraph obtained in Stage 2

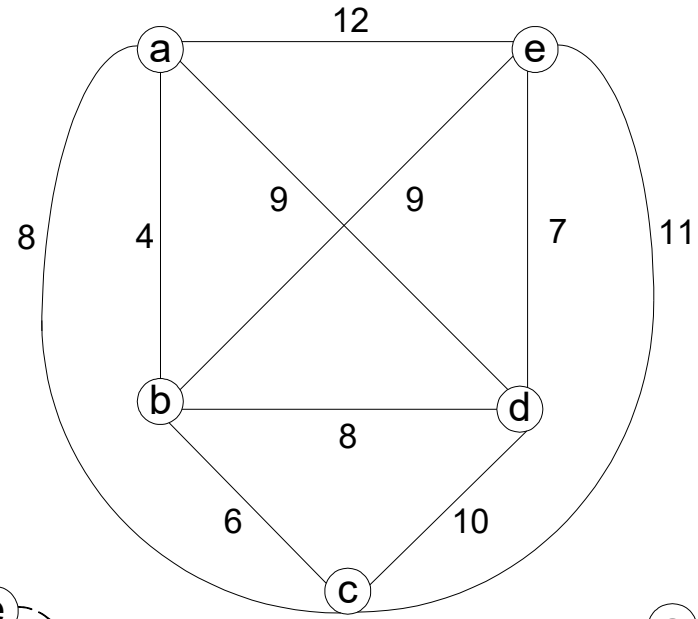
Stage 3: Create a tour from the path constructed in Stage 2 by making shortcuts to avoid visiting intermediate vertices more than once

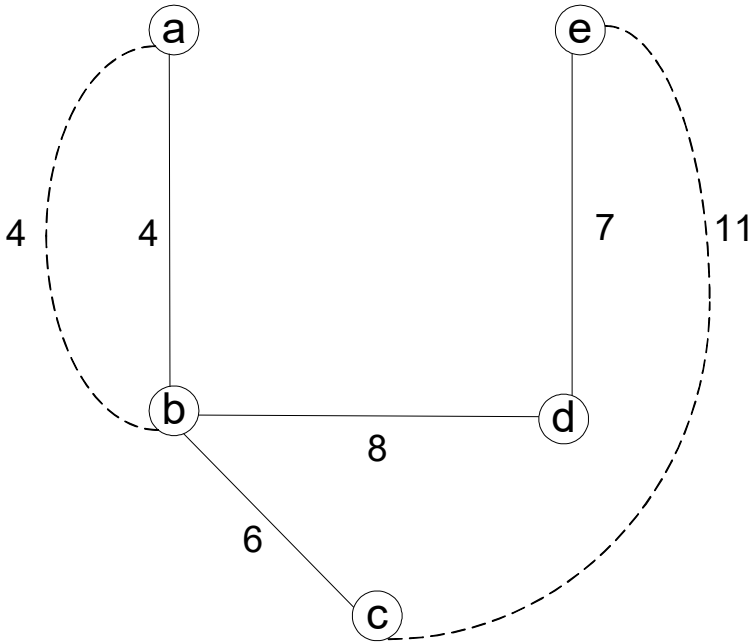
$R_A = \infty$ for general instances, but it tends to produce better tours than the twice-around-the-minimum-tree alg.

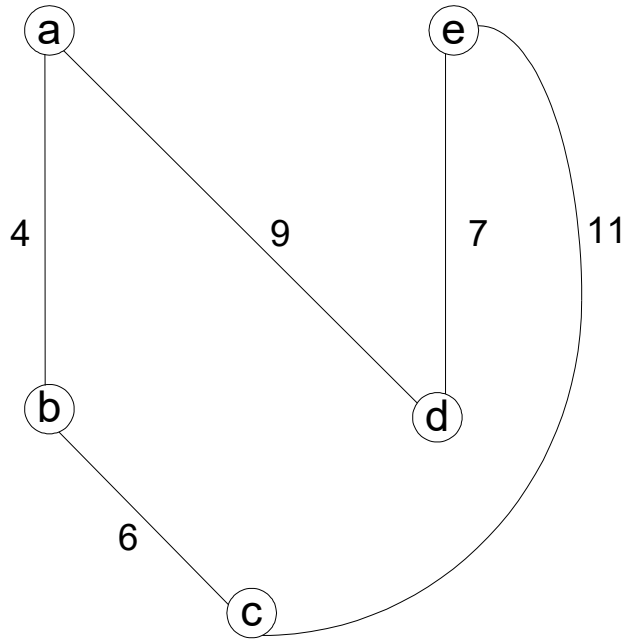


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: Christofides Algorithm







Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Euclidean Instances

Theorem If $P \neq NP$, there exists no approximation algorithm for TSP with a finite performance ratio.

Definition An instance of TSP is called *Euclidean*, if its distances satisfy two conditions:

1. *symmetry* $d[i, j] = d[j, i]$ for any pair of cities i and j
2. *triangle inequality* $d[i, j] \leq d[i, k] + d[k, j]$ for any cities i, j, k

For Euclidean instances:

approx. tour length / optimal tour length $\leq 0.5(\lceil \log_2 n \rceil + 1)$ for nearest neighbor and multifragment heuristic;

approx. tour length / optimal tour length ≤ 2 for twice-around-the-tree;

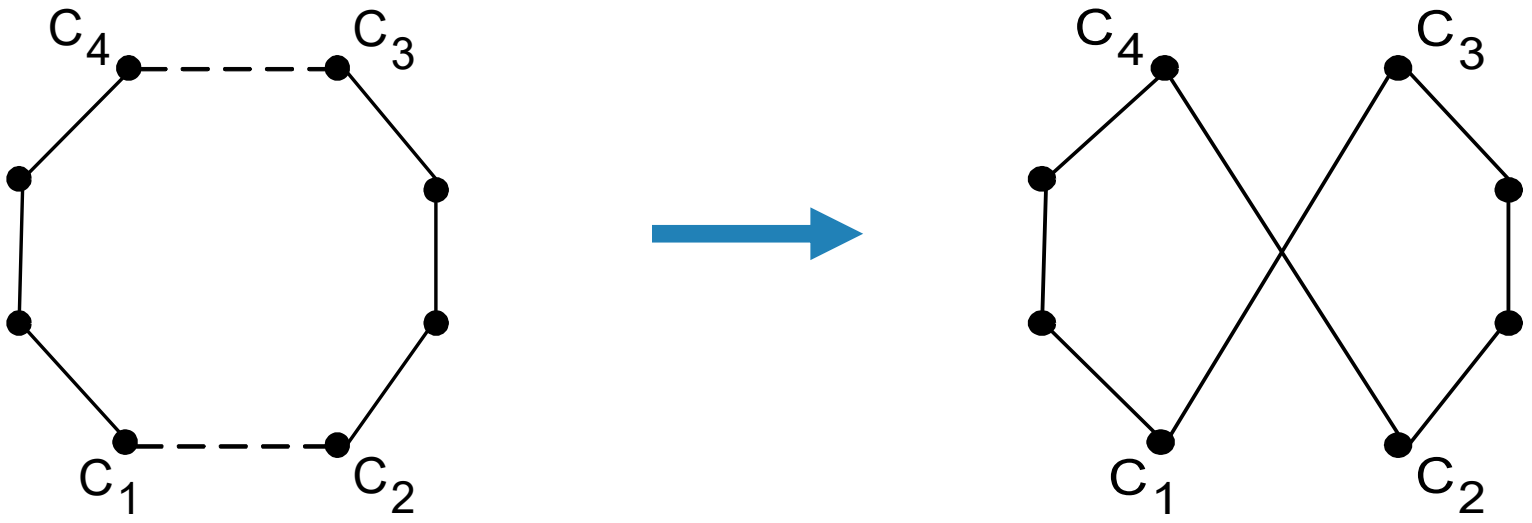
approx. tour length / optimal tour length ≤ 1.5 for Christofides

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Local Search Heuristics for TSP

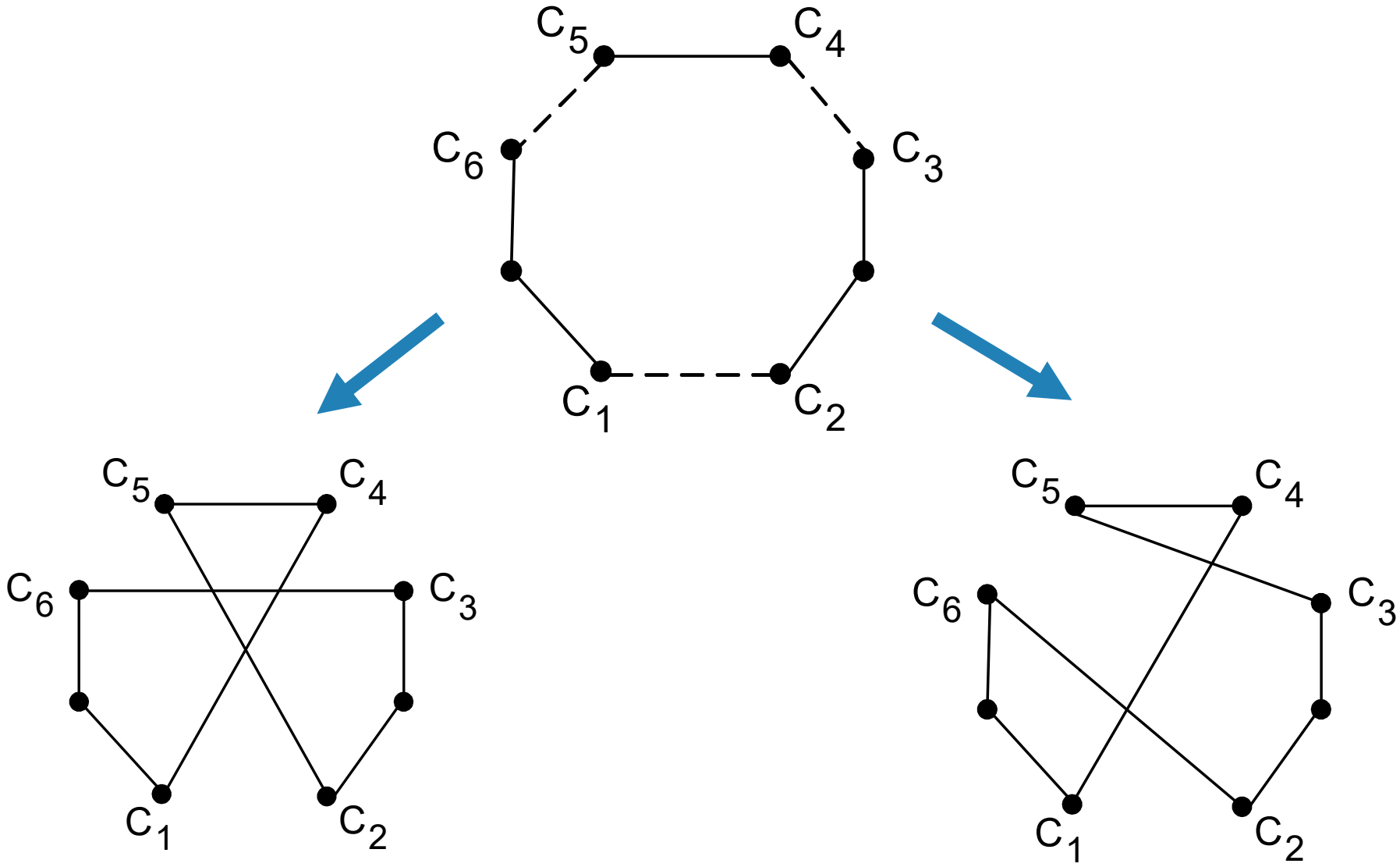
Start with some initial tour (e.g., nearest neighbor). On each iteration, explore the current tour's neighborhood by exchanging a few edges in it. If the new tour is shorter, make it the current tour; otherwise consider another edge change. If no change yields a shorter tour, the current tour is returned as the output.

Example of a 2-change



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example of a 3-change



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

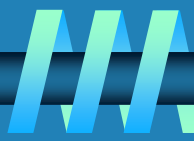
Empirical Data for Euclidean Instances

TABLE 12.1 Average tour quality and running times for various heuristics on the 10,000-city random uniform Euclidean instances [Joh02]

Heuristic	% excess over the Held-Karp bound	Running time (seconds)
nearest neighbor	24.79	0.28
multifragment	16.42	0.20
Christofides	9.81	1.04
2-opt	4.70	1.41
3-opt	2.88	1.50
Lin-Kernighan	2.00	2.06

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Greedy Algorithm for Knapsack Problem



Step 1: Order the items in decreasing order of relative values:
$v_1/w_1 \geq \cdots \geq v_n/w_n$

Step 2: Select the items in this order skipping those that don't fit into the knapsack

Example: The knapsack's capacity is 16

item	weight	value	v/w
1	2	$40	20
2	5	$30	6
3	10	$50	5
4	5	$10	2

rem. cap.

pick item 1 — 14

pick item 2 — 9

can't pick item 3

pick item 4 — 4

value: 80

optimal: 90

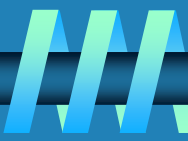
accuracy ratio: 90/80 = 9/8.

Accuracy

- R_A is unbounded (e.g., $n = 2$, $C = m$, $w_1=1$, $v_1=2$, $w_2=m$, $v_2=m$)
- yields exact solutions for the continuous version

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Approximation Scheme for Knapsack Problem

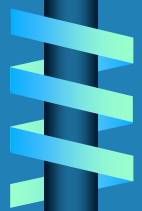


Step 1: Order the items in decreasing order of relative values:
$v_1/w_1 \geq \cdots \geq v_n/w_n$

Step 2: For a given integer parameter k, $0 \leq k \leq n$, generate all subsets of k items or less and for each of those that fit the knapsack, add the remaining items in decreasing order of their value to weight ratios

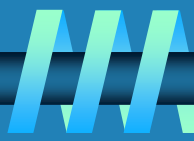
Step 3: Find the most valuable subset among the subsets generated in Step 2 and return it as the algorithm's output

- **Accuracy:** $f(s^*) / f(s_a) \leq 1 + 1/k$ for any instance of size n
- **Time efficiency:** $O(kn^{k+1})$
- There are *fully polynomial schemes*: algorithms with polynomial running time as functions of both n and k



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Bin Packing Problem: First-Fit Algorithm



***First-Fit* (*FF*) Algorithm**: Consider the items in the order given and place each item in the first available bin with enough room for it; if there are no such bins, start a new one

Example: $n = 4,\ s_1 = 0.4,\ s_2 = 0.2,\ s_3 = 0.6,\ s_4 = 0.7$

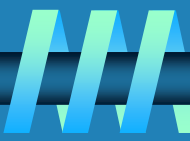
Accuracy

- Number of extra bins never exceeds optimal by more than 70% (i.e., $R_A \leq 1.7$)
- Empirical average-case behavior is much better. (In one experiment with 128,000 bins, the relative error was found to be no more than 2%.)



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Bin Packing: First-Fit Decreasing Algorithm



First-Fit Decreasing (FFD) Algorithm: Sort the items in decreasing order (i.e., from the largest to the smallest). Then proceed as above by placing an item in the first bin in which it fits and starting a new bin if there are no such bins

Example: $n = 4,\ s_1 = 0.4,\ s_2 = 0.2,\ s_3 = 0.6,\ s_4 = 0.7$

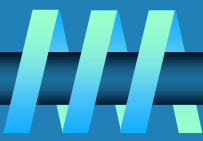
Accuracy

- Number of extra bins never exceeds optimal by more than 50% (i.e., $R_A \leq 1.5$)
- Empirical average-case behavior is much better, too



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

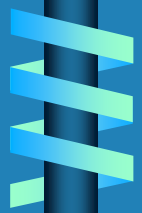
Numerical Algorithms



Numerical algorithms **concern with solving mathematical problems such as**

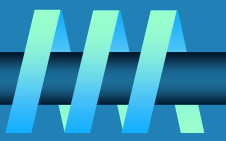
- **evaluating functions (e.g., $\sqrt{x}$, e^x, ln x, sin *x*)**
- **solving nonlinear equations**
- **finding extrema of functions**
- **computing definite integrals**

Most such problems are of "continuous" nature and can be solved only approximately



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Principal Accuracy Metrics



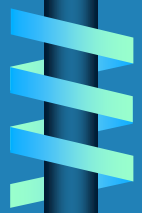
- ***Absolute error*** **of approximation (of α^* by α)**

$$|\alpha - \alpha^*|$$

- ***Relative error*** **of approximation (of α^* by α)**

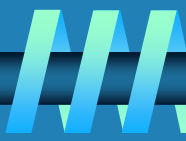
$$|\alpha - \alpha^*| / |\alpha^*|$$

  - **undefined for $\alpha^* = 0$**
  - **often quoted in %**



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Two Types of Errors



- ***truncation errors***
  - **Taylor's polynomial approximation**

$$e^x \approx 1 + x + x^2/2! + \cdots + x^n/n!$$

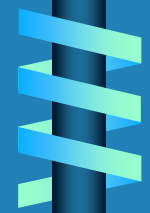
absolute error $\leq M\,|x|^{n+1}/(n+1)!$ **where** $M = \max e^t$ **for** $0 \leq t \leq x$

  - **composite trapezoidal rule**

$$\int_a^b f(x)dx \approx (h/2)\,[f(a) + 2\Sigma_{1\leq=i\leq=n-1} f(x_i) + f(b)],\quad h = (b-a)/n$$

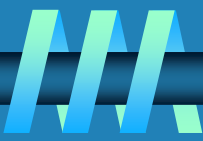
absolute error $\leq (b-a)h^2\,M_2 / 12$ **where** $M_2 = \max |f''(x)|$ **for** $a \leq x \leq b$

- ***round-off errors***



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Solving Quadratic Equation



Quadratic equation $ax^2 + bx + c = 0$ $(a \neq 0)$

$$x_{1,2} = (-b \pm \sqrt{D})/2a \quad \text{where } D = b^2 - 4ac$$

Problems:

- computing square root

  use Newton's method: $x_{n+1} = 0.5(x_n + D/x_n)$

- subtractive cancellation

  use alternative formulas (see p. 411)

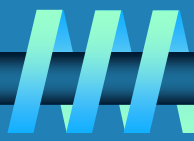
  use double precision for $D = b^2 - 4ac$

- other problems (overflow, etc.)



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Notes on Solving Nonlinear Equations



- There exist no formulas with arithmetic ops. and root extractions for roots of polynomials

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0 = 0 \quad \text{of degree } n \geq 5$$

- Although there exist special methods for approximating roots of polynomials, one can also use general methods for

$$f(x) = 0$$

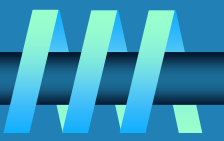
- Nonlinear equation $f(x) = 0$ can have one, many, infinitely many, and no roots at all

- Useful:
  - sketch graph of $f(x)$
  - separate roots



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Three Classic Methods



Three classic methods for solving nonlinear equation

$$f(x) = 0$$

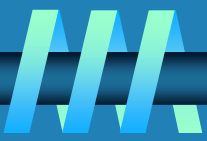
in one unknown:

- **bisection method**
- **method of false position (regula falsi)**
- **Newton's method**



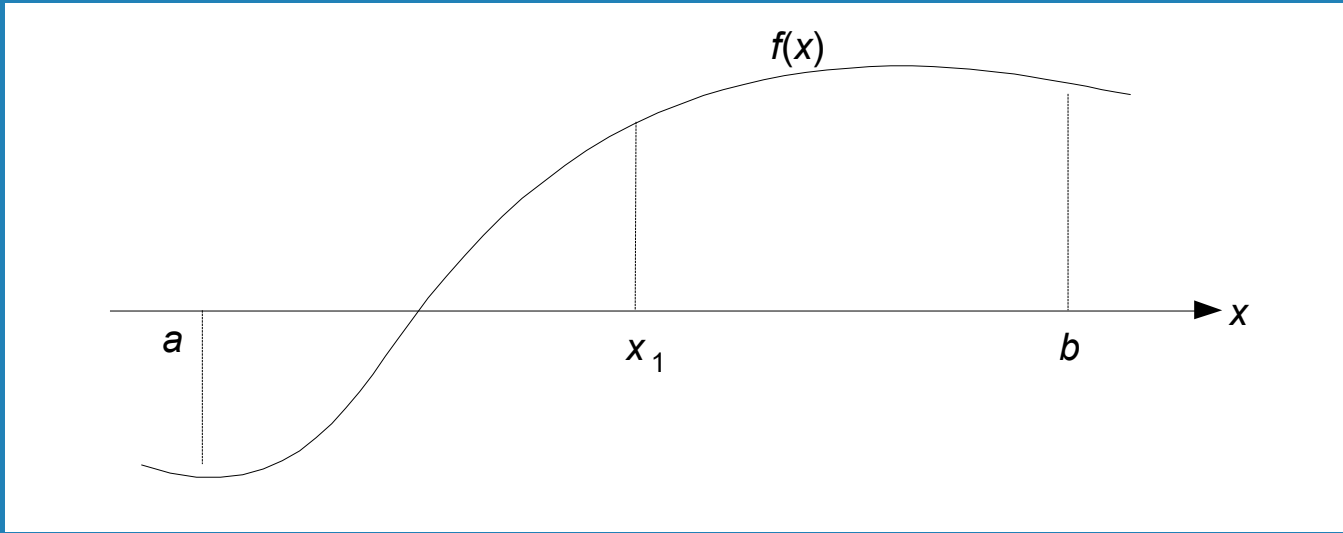
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Bisection Method



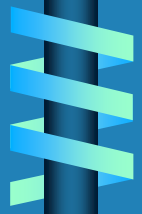
Based on

- **Theorem: If $f(x)$ is continuous on $a \le x \le b$ and $f(a)$ and $f(b)$ have opposite signs, then $f(x) = 0$ has a root on $a < x < b$**
- **binary search idea**



Approximations x_n are middle points of shrinking segments

- **$|x_n - x^*| \le (b - a)/2^n$**
- **x_n always converges to root x^* but slower compared to others**



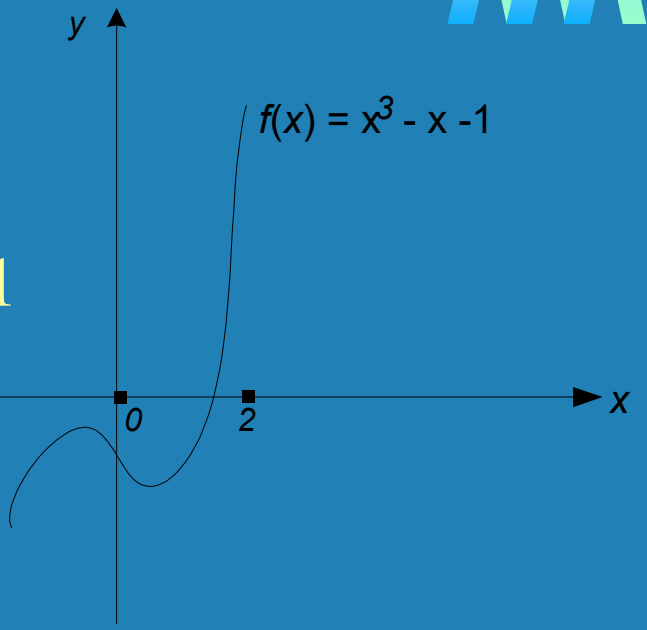
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example of Bisection Method Application

Find the root of

$$x^3 - x - 1 = 0$$

with the absolute error not larger than 0.01



n	a_n	b_n	x_n	$f(x_n)$
1	0.0-	2.0+	1.0	-1.0
2	1.0-	2.0+	1.5	0.875
3	1.0-	1.5+	1.25	-0.296875
4				
5				
6				
7				
8	1.3125-	1.328125+	1.3203125	-0.018711

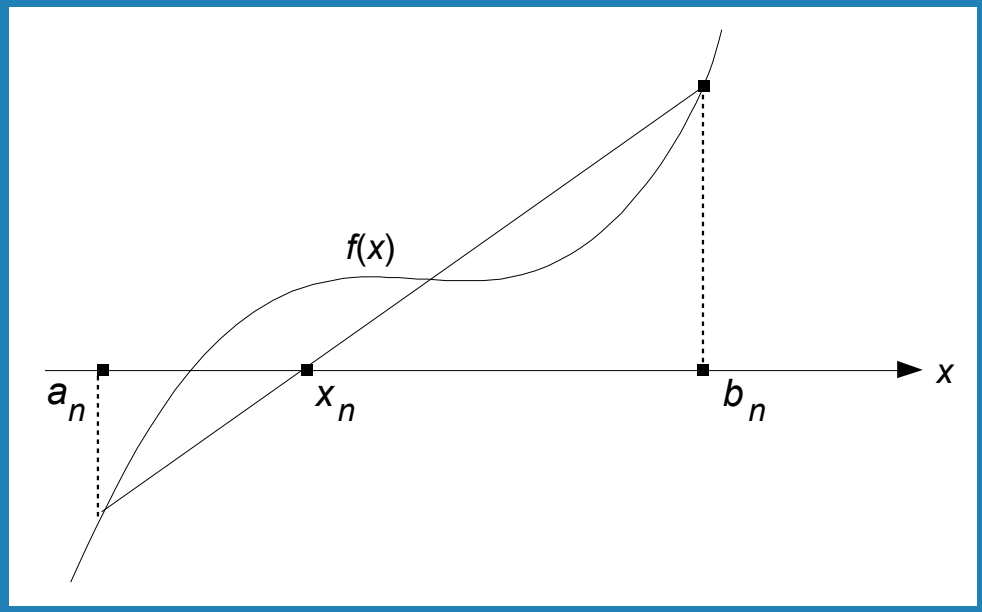
$x \approx 1.3203125$

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Method of False Position



Similar to bisection method but uses x-intercept of line through $(a, f(a))$ and $(b, f(b))$ instead of middle point of $[a,b]$



Approximations x_n are computed by the formula

$$x_n = [a_n f(b_n) - b_n f(a_n)] / [f(b_n) - f(a_n)]$$

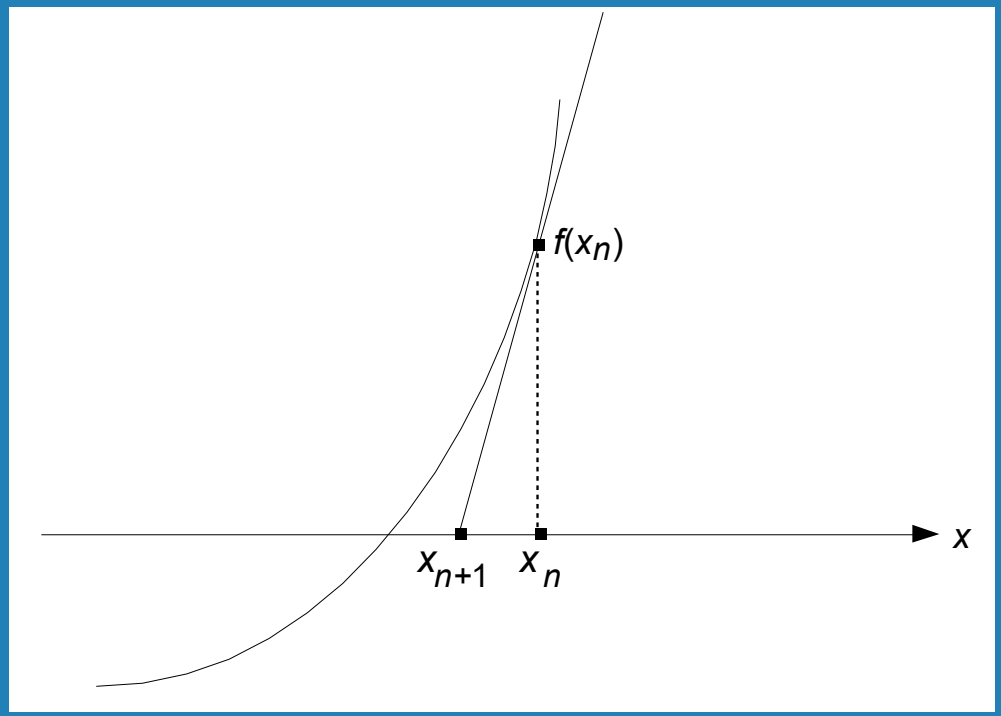
- Normally x_n converges faster than bisection method sequence but slower than Newton's method sequence

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Newton's Method

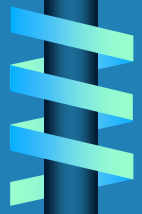


Very fast method in which x_n's are x-intercepts of tangent lines to the graph of $f(x)$



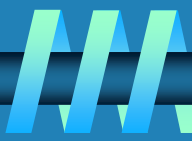
Approximations x_n are computed by the formula

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

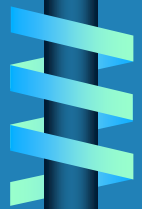
Notes on Newton's Method



- Normally, approximations x_n converge to root very fast but can diverge with a bad choice of initial approximation x_0
- Yields a very fast method for computing square roots

$$x_{n+1} = 0.5(x_n + D/x_n)$$

- Can be generalized to much more general equations



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 12 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.